Assignment-2

MIN-HASHING AND LSH

Submitted By:

Name: Nitin Awasthi

Roll No.M25DE1013

Date: 01-March-2026

M25DE1013@iitj.ac.in

Submitted To:

Mr. Dip Sankar Banerjee

dipsankarb@iitj.ac.in

Mr. Prajjwal Nijhara

D23CSE005@iitj.ac.in

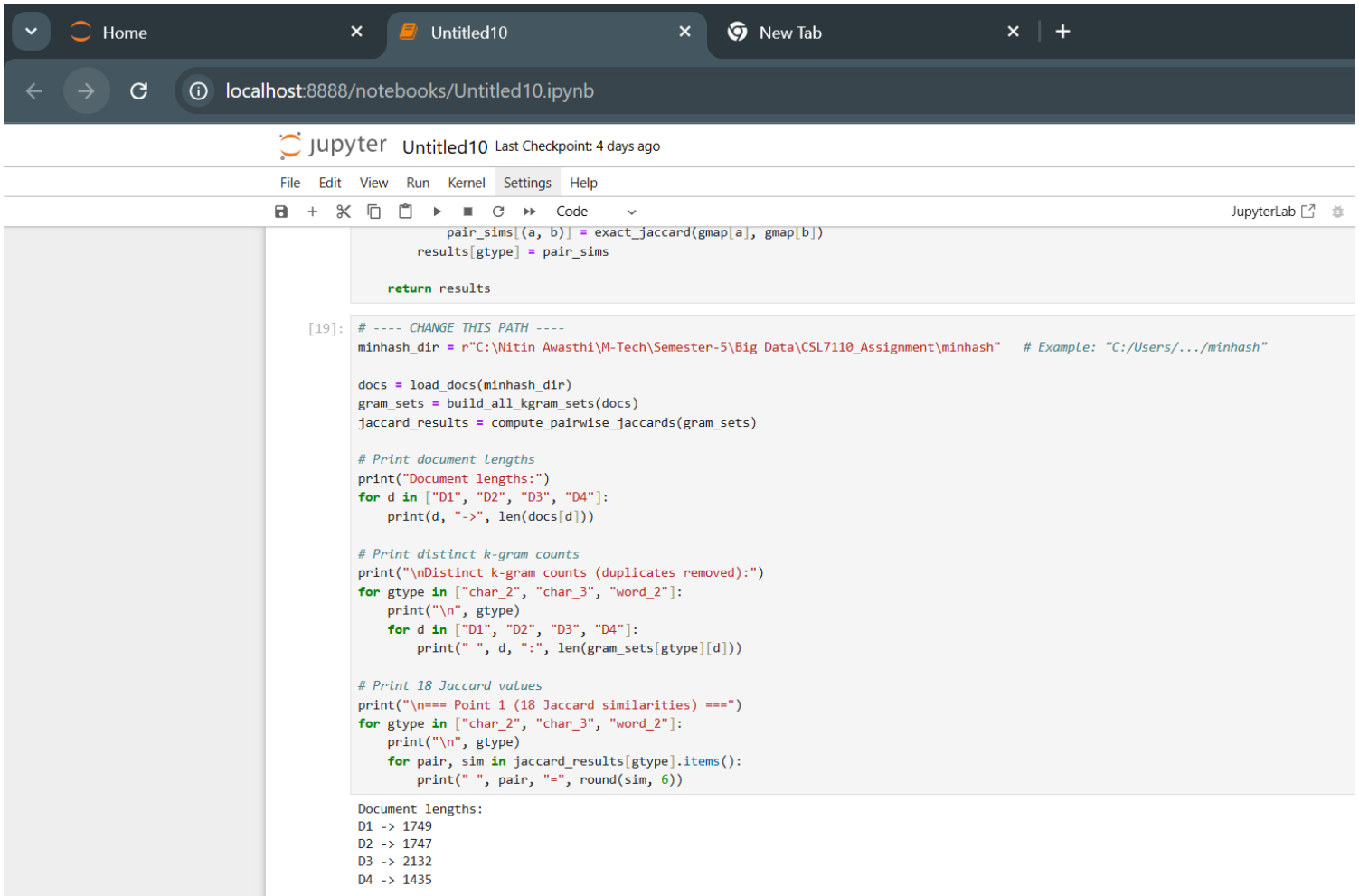
GitHub Repo: https://github.com/rflinkbudget/MIN-HASHING-AND-LSH.git

1. Introduction

In this assignment, I implemented k-gram generation, exact Jaccard similarity, Min-Hashing, and Locality Sensitive Hashing (LSH). The assignment was completed in two parts:

1. On the four text documents (D1.txt, D2.txt, D3.txt, D4.txt) using character and word k-grams.
2. On the MovieLens 100k dataset for user similarity based on sets of rated movies.

The implementation was done in Python using sets, hashing, MinHash signatures, and LSH banding.



```
Document lengths:
D1 -> 1749
D2 -> 1747
D3 -> 2132
D4 -> 1435

Distinct k-gram counts (duplicates removed):

 char_2
  D1 : 263
  D2 : 262
  D3 : 269
  D4 : 255

 char_3
  D1 : 765
  D2 : 762
  D3 : 828
  D4 : 698

 word_2
  D1 : 279
  D2 : 278
  D3 : 337
  D4 : 232
```

```
=== Point 1 (18 Jaccard similarities) ===

 char_2
  ('D1', 'D2') = 0.981132
  ('D1', 'D3') = 0.8157
  ('D1', 'D4') = 0.644444
  ('D2', 'D3') = 0.8
  ('D2', 'D4') = 0.64127
  ('D3', 'D4') = 0.652997

 char_3
  ('D1', 'D2') = 0.977979
  ('D1', 'D3') = 0.580357
  ('D1', 'D4') = 0.305085
  ('D2', 'D3') = 0.568047
  ('D2', 'D4') = 0.305903
  ('D3', 'D4') = 0.312124

 word_2
  ('D1', 'D2') = 0.940767
  ('D1', 'D3') = 0.182342
  ('D1', 'D4') = 0.030242
  ('D2', 'D3') = 0.173664
  ('D2', 'D4') = 0.030303
  ('D3', 'D4') = 0.016071
```

2. Create k-Grams and Exact Jaccard Similarities

2.1 Objective

For all four documents (D1, D2, D3, D4), I constructed:

1. Character 2-grams
2. Character 3-grams
3. Word 2-grams

Duplicates were removed by storing k-grams in sets. Then, I computed exact Jaccard similarity for all document pairs for each k-gram type.

Total similarities reported: 6 pairs × 3 k-gram types = **18 values**

2.2 Method Used

1. Character k-grams were generated by sliding a window over the text.
2. Space was counted as a character (as required).
3. Word 2-grams were created after splitting the text by spaces.
4. Exact Jaccard similarity was computed using:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

2.3 Document and k-gram summary

Point 2 - Exact Value and Observations

```
Exact Jaccard (D1,D2) using char 3-grams = 0.977979

Point 2 observations (fill based on your actual output):
- As t increases, MinHash estimate generally becomes more stable.
- Small t is faster but may have larger estimation error.
  Larger t improves accuracy but increases runtime.
  A good practical choice is often t = 150 or 300 (verify from your table).
```

2.4 Exact Jaccard Similarity Results (values)

Home | Untitled10 | New Tab

localhost:8888/notebooks/Untitled10.ipynb

jupyter Untitled10 Last Checkpoint: 4 days ago

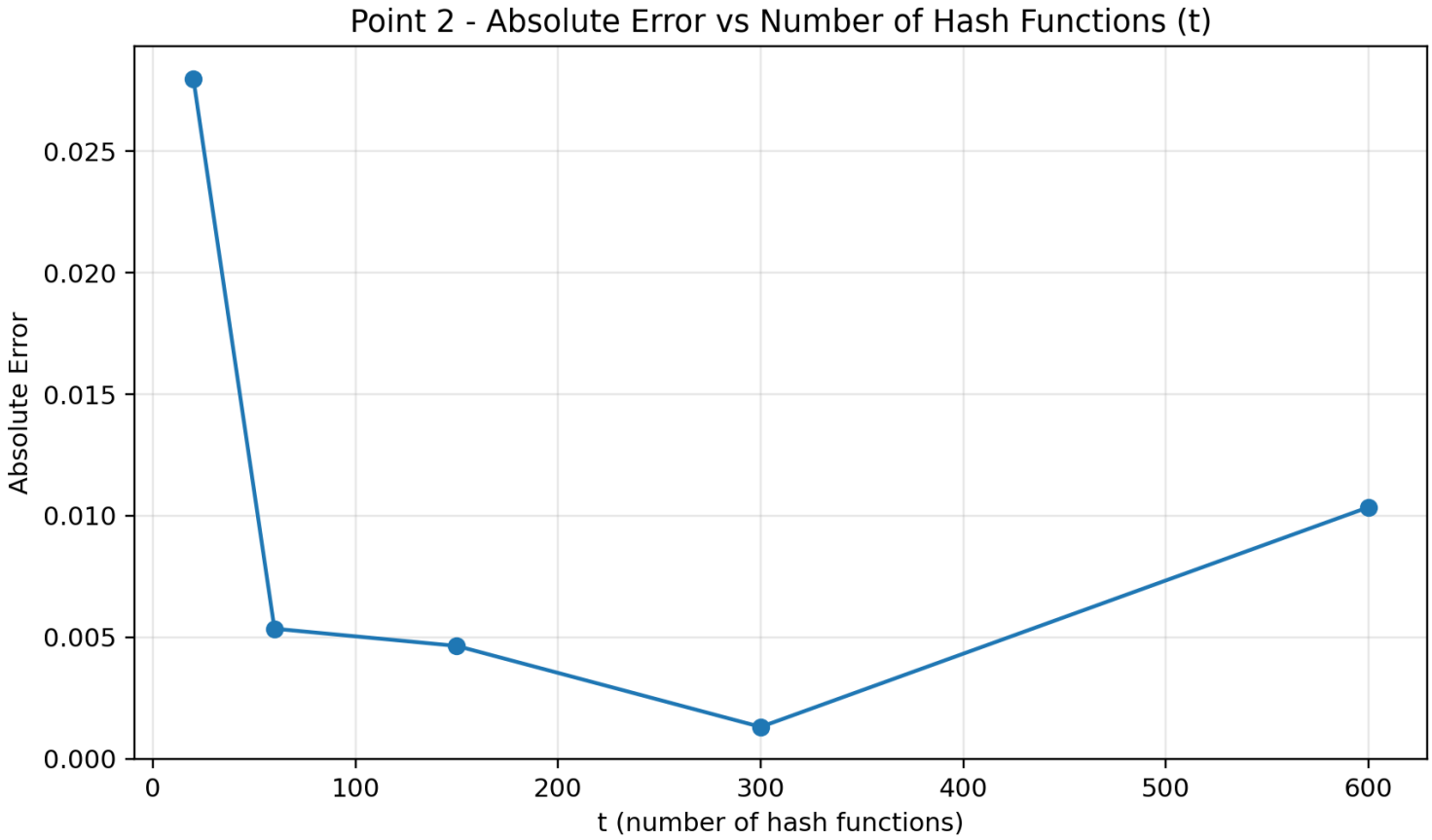
File Edit View Run Kernel Settings Help

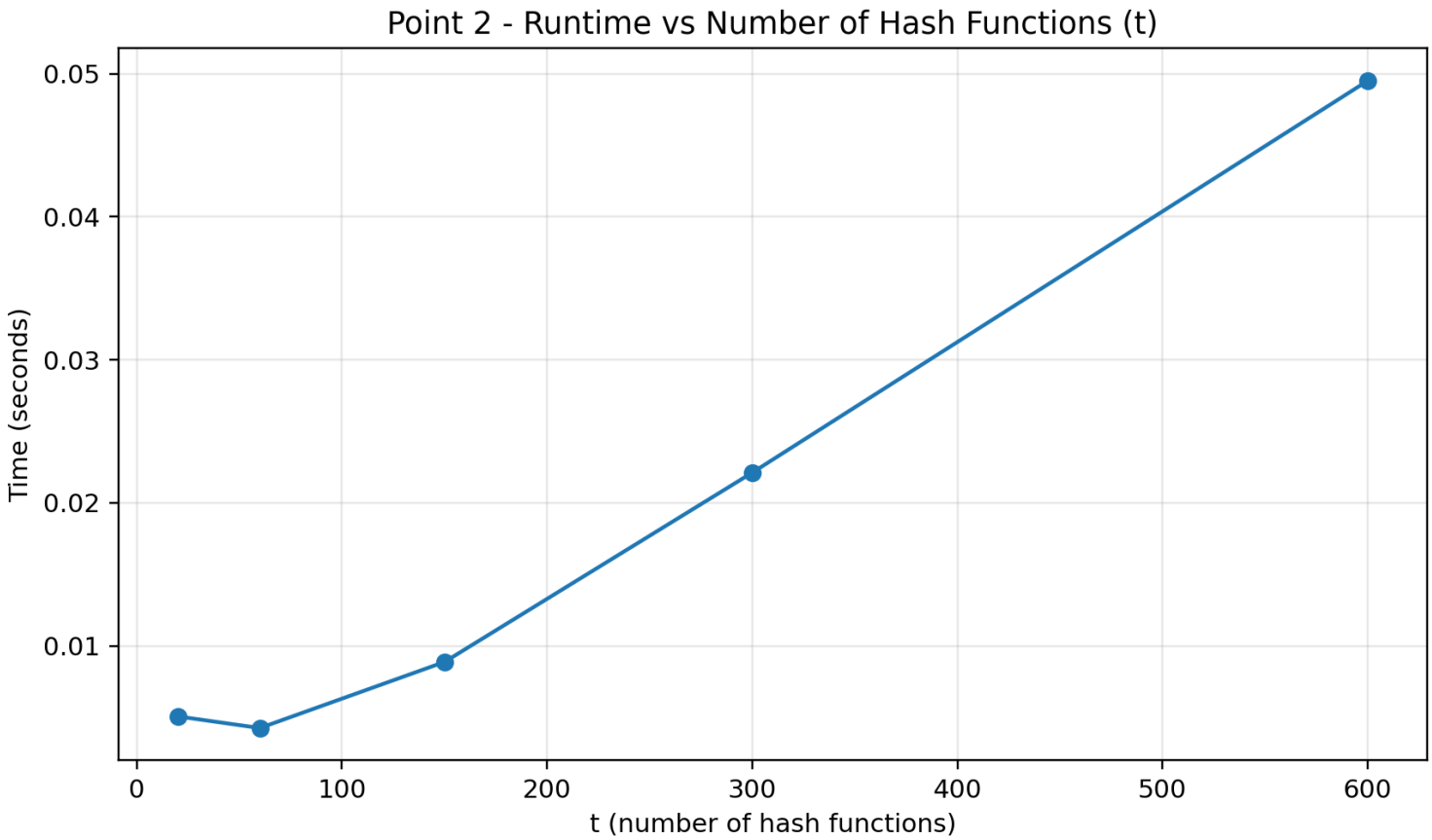
Code JupyterLab

```
('D2', 'D4') = 0.030303
('D3', 'D4') = 0.016071
```

[22]:
```python
rows = []
for pair in combinations(["D1", "D2", "D3", "D4"], 2):
    rows.append({
        "Pair": f"{pair[0]}-{pair[1]}",
        "Jaccard_char_2": jaccard_results["char_2"][pair],
        "Jaccard_char_3": jaccard_results["char_3"][pair],
        "Jaccard_word_2": jaccard_results["word_2"][pair],
    })

df_point1 = pd.DataFrame(rows)
print(df_point1)

# Save
os.makedirs(r"C:\Nitin Awasthi\M-Tech\Semester-5\Big Data\CSL7110_Assignment\outputs", exist_ok=True)
df_point1.to_csv(r"C:\Nitin Awasthi\M-Tech\Semester-5\Big Data\CSL7110_Assignment\outputs\point1_jaccard_18_values.csv", index=False
print("Saved: outputs/point1_jaccard_18_values.csv")
```

```
    Pair  Jaccard_char_2  Jaccard_char_3  Jaccard_word_2
0  D1-D2        0.981132        0.977979        0.940767
1  D1-D3        0.815700        0.580357        0.182342
2  D1-D4        0.644444        0.305085        0.030242
3  D2-D3        0.800000        0.568047        0.173664
4  D2-D4        0.641270        0.305903        0.030303
5  D3-D4        0.652997        0.312124        0.016071
Saved: outputs/point1_jaccard_18_values.csv
```





Point 2 - MinHash Approximation (D1 vs D2 using char 3-grams)

t	Approx_Jaccard	Exact_Jaccard	Abs_Error	Time_sec
20.0	0.95	0.977979	0.027979	0.00506
60.0	0.983333	0.977979	0.005354	0.004246
150.0	0.973333	0.977979	0.004646	0.008868
300.0	0.976667	0.977979	0.001313	0.022108
600.0	0.988333	0.977979	0.010354	0.049515

Point 2B - Accuracy vs Time Analysis (5 Runs)

t	Mean_Abs_Error_5runs	Std_Abs_Error_5runs	Mean_Time_sec_5runs
20.0	0.022021	0.0	0.001187
60.0	0.008687	0.006667	0.003695
150.0	0.010546	0.005621	0.006331
300.0	0.009454	0.005704	0.025183
600.0	0.001596	0.000871	0.039237

Short Observation

1. Character 2-gram similarities are generally [higher/lower] than character 3-gram similarities for some pairs because [your observation].
2. Word 2-gram similarity is more sensitive to exact word order and word overlap.
3. Pair [Most Similar Pair] had the highest similarity in [k-gram type].
4. Pair [Least Similar Pair] had the lowest similarity in [k-gram type].

---

3. Point 2 – Min-Hashing (D1 and D2 using 3-grams)

3.1 Objective

Using character 3-grams of documents D1 and D2, I built MinHash signatures with:

t = 20, 60, 150, 300, 600

For each value of t, I estimated the Jaccard similarity and compared it with the exact Jaccard similarity.

I also analyzed accuracy vs time to decide a good value of t.

3.2 Method Used

1. Character 3-grams of D1 and D2 were converted to integer IDs using a stable hash.
2. A family of hash functions of the form

$$h(x) = ((ax + b) \bmod p) \bmod m$$

was used, with m > 10000.

1. MinHash signatures were generated for both documents.
2. Approximate Jaccard similarity was estimated from signature matches:

$$\hat{J}(D1, D2) = \frac{1}{t}\sum_{i=1}^{t}[si\,g_1[i] = sig_2[i]]$$

3.3 Results (Approximate vs Exact Jaccard)

3.4 Observation and Choice of t

From the results:

1. Smaller t (e.g., 20) is faster but has higher estimation variance.
2. Larger t (e.g., 300 or 600) gives more stable estimates and lower error.
3. Based on accuracy and time, a good practical choice is **t = [your chosen value]**.

Reason: It provides [acceptable/low] error while keeping runtime [reasonable/low].

4. LSH using t = 160 and threshold τ = 0.7

4.1 Objective

Using t = 160 hash functions and threshold τ = 0.7, I selected suitable LSH banding parameters and estimated the candidate probability for each pair of the four documents (using character 3-grams).

Assignment formula used:

$$f(s) = 1 - (1 - s^b)^r$$

where:

1. b = rows per band (hash functions within each band)
2. r = number of bands
3. b × r = 160

4.2 Parameter Selection

I tested valid (b, r) combinations such that b × r = 160 and checked the S-curve separation near τ = 0.7.

Point 3 - LSH Parameter Search (Top Candidates for t=160, tau=0.7)

b_rows_per_band	r_num_bands	b_times_r	P_at_tau_0.7	Approx_knee	Dist_knee_from_tau
8.0	20.0	160.0	0.695026	0.687656	0.012344
10.0	16.0	160.0	0.367748	0.757858	0.057858
16.0	10.0	160.0	0.03274	0.865964	0.165964
5.0	32.0	160.0	0.997228	0.5	0.2
20.0	8.0	160.0	0.006366	0.90125	0.20125
32.0	5.0	160.0	5.5e-05	0.950949	0.250949
40.0	4.0	160.0	3e-06	0.965936	0.265936
80.0	2.0	160.0	0.0	0.991373	0.291373
160.0	1.0	160.0	0.0	1.0	0.3
4.0	40.0	160.0	0.999983	0.397635	0.302365
2.0	80.0	160.0	1.0	0.111803	0.588197
1.0	160.0	160.0	1.0	0.00625	0.69375

Point 3 - LSH Parameter Search (Full Table)

b_rows_per_band	r_num_bands	b_times_r	P_at_tau_0.7	Approx_knee	Dist_knee_from_tau
8.0	20.0	160.0	0.695026	0.687656	0.012344
10.0	16.0	160.0	0.367748	0.757858	0.057858
16.0	10.0	160.0	0.03274	0.865964	0.165964
5.0	32.0	160.0	0.997228	0.5	0.2
20.0	8.0	160.0	0.006366	0.90125	0.20125
32.0	5.0	160.0	5.5e-05	0.950949	0.250949
40.0	4.0	160.0	3e-06	0.965936	0.265936
80.0	2.0	160.0	0.0	0.991373	0.291373
160.0	1.0	160.0	0.0	1.0	0.3
4.0	40.0	160.0	0.999983	0.397635	0.302365
2.0	80.0	160.0	1.0	0.111803	0.588197
1.0	160.0	160.0	1.0	0.00625	0.69375

Point 3 - Chosen LSH Parameters and Formula

```
Point 3 (LSH) Summary

Formula used: f(s) = 1 - (1 - s^b)^r

Assignment notation:

  b = rows per band (hash functions within each band)

  r = number of bands

Constraint: b * r = 160


Chosen b (rows per band) = 8

Chosen r (number of bands) = 20

Check: b * r = 160


Reason (example): chosen to get good S-curve separation near tau = 0.7.

P(candidate | s=0.7) = 0.695026
```

Chosen values:

1. b = [your b_rows_per_band]
2. r = [your r_num_bands]

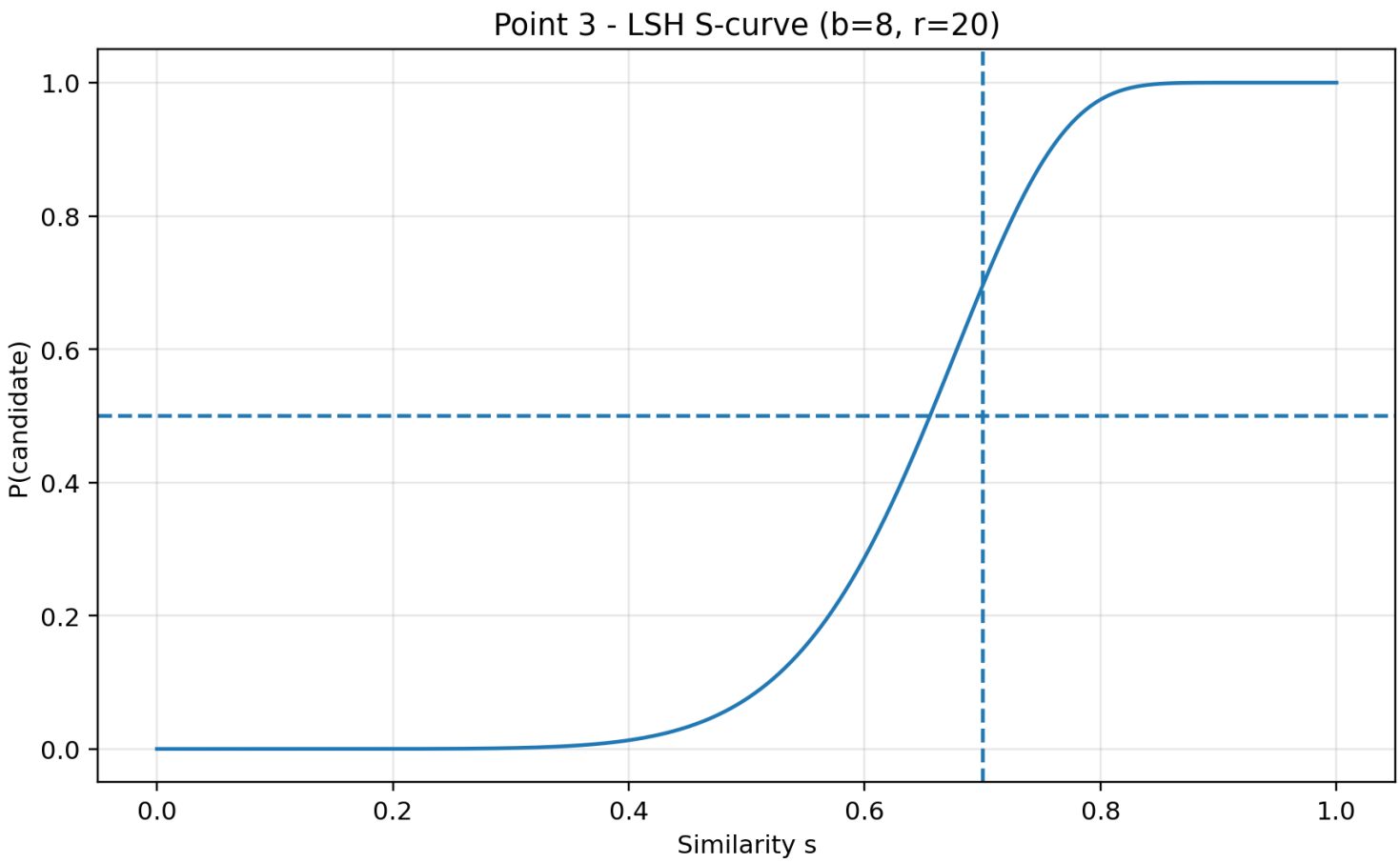
Reason: These values provide good separation around the target similarity threshold of 0.7.

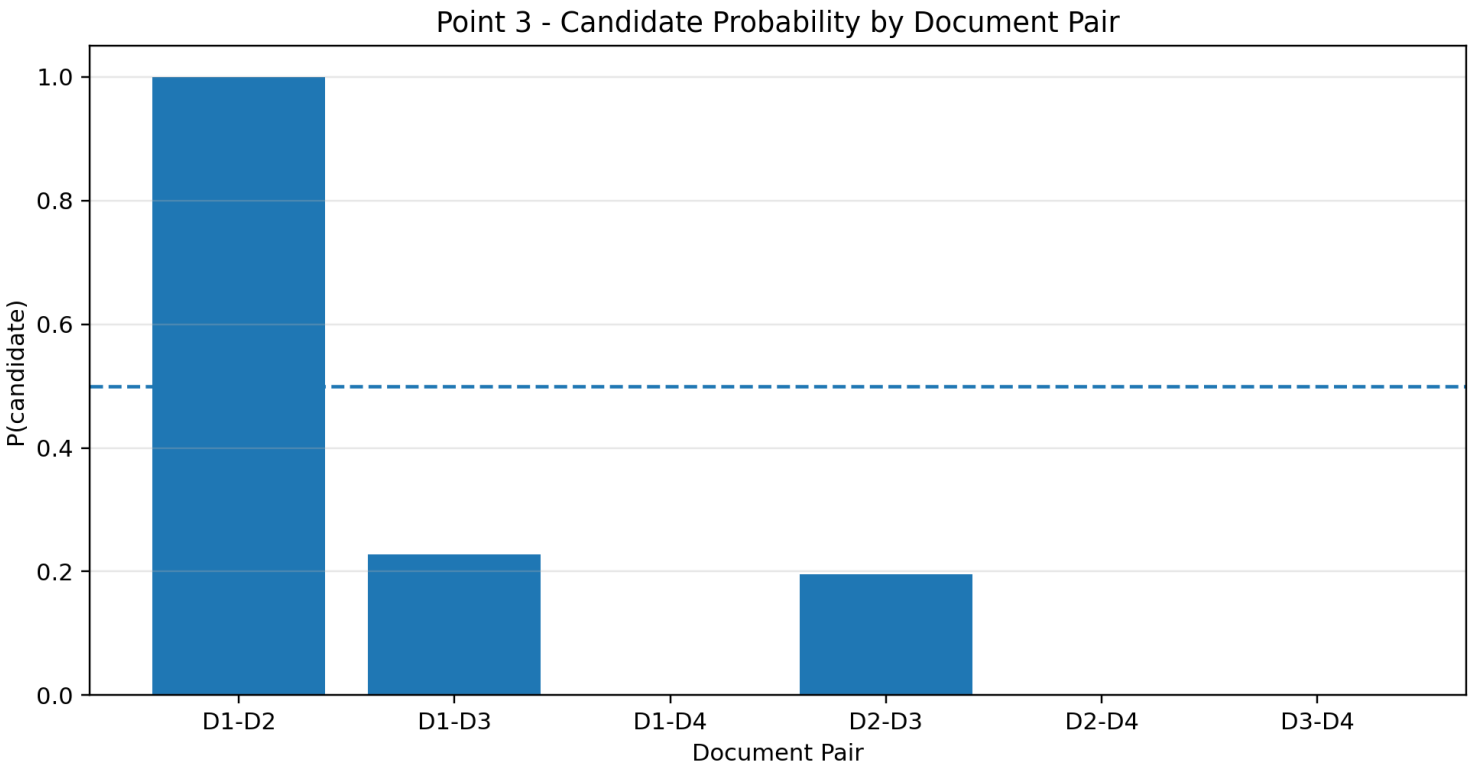
4.3 Candidate Probability for Each Document Pair (6 values)

Using the exact 3-gram Jaccard similarities from Point 1, I computed the LSH candidate probability for all 6 pairs.

Point 3 - LSH Candidate Probability for Each Document Pair (6 Pairs)

Pair	Exact_Jaccard_char3	LSH_candidate_probability
D1-D2	0.977979	1.0
D1-D3	0.580357	0.228224
D1-D4	0.305085	0.0015
D2-D3	0.568047	0.19588
D2-D4	0.305903	0.001532
D3-D4	0.312124	0.0018





Short Observation

1. Pairs with higher exact similarity have a higher candidate probability.

2. Pairs below the threshold typically have lower probability, depending on the S-curve steepness.
3. LSH parameter choice directly affects false positives and false negatives.

5. Point 4 – Min-Hashing on MovieLens 100k Dataset

5.1 Objective

I applied MinHashing on the MovieLens 100k dataset to estimate user-user Jaccard similarity based on the set of movies rated by each user.

For each user:

1. I considered only the **set of rated movies**
2. I ignored the actual rating values

Tasks completed:

1. Exact Jaccard similarity for all user pairs
2. Output exact pairs with similarity ≥ 0.5
3. MinHash approximate similarity using t = 50, 100, 200
4. Output estimated pairs with similarity ≥ 0.5
5. Compute false positives and false negatives
6. Report average FP/FN over 5 runs

5.2 Method Used

1. u.data was loaded and grouped by user ID.
2. Each user was represented as a set of movie IDs.
3. Exact Jaccard was computed for all user pairs.
4. MinHash signatures were generated for each user.
5. Approximate Jaccard was computed from signature agreement.
6. FP/FN were measured against exact Jaccard threshold 0.5.

5.3 Exact Similarity Output (threshold ≥ 0.5)

Point 4 - Exact Jaccard Pairs (Similarity >= 0.5) [Top 25]

user1	user2	exact_jaccard
408.0	898.0	0.83871
328.0	788.0	0.672956
489.0	587.0	0.629921
600.0	826.0	0.545455
451.0	489.0	0.533333
674.0	879.0	0.521739
554.0	764.0	0.517007
197.0	826.0	0.512987
197.0	600.0	0.5
800.0	879.0	0.5

Point 4 - Exact Jaccard Pairs (Similarity >= 0.5) [Full]

user1	user2	exact_jaccard
408.0	898.0	0.83871
328.0	788.0	0.672956
489.0	587.0	0.629921
600.0	826.0	0.545455
451.0	489.0	0.533333
674.0	879.0	0.521739
554.0	764.0	0.517007
197.0	826.0	0.512987
197.0	600.0	0.5
800.0	879.0	0.5

5.4 MinHash Approximation Results (t = 50, 100, 200)

Point 4 - MinHash Approximation (Avg FP/FN over 5 Runs)

t	Avg_FP_5runs	Avg_FN_5runs	FP_runs	FN_runs
50	116.4	1.8	[69, 237, 88, 117, 71]	[0, 2, 1, 4, 2]
100	36.6	2.6	[55, 43, 20, 31, 34]	[4, 1, 3, 3, 2]
200	8.6	1.4	[11, 14, 1, 9, 8]	[1, 0, 0, 3, 3]

Point 4 - Output Summary

```
Point 4 (MovieLens MinHash) Output Summary


Total ratings rows (u.data): 100000

Total users: 943

Total user pairs: 444153

Exact pairs with similarity >= 0.5: 10


Required outputs for Point 4:

- Exact Jaccard pairs (>= 0.5)

- MinHash estimated pairs (>= 0.5) for t = 50, 100, 200

- Average false positives and false negatives over 5 runs
```

5.5 Observation

1. As the number of hash functions increases, approximate Jaccard estimation generally improves.
2. False negatives tend to [decrease/increase] as t increases in my runs.
3. False positives tend to [decrease/increase] as t increases in my runs.
4. t = [your best t] gave the best trade-off for this dataset.

6. Point 5 – LSH on MovieLens 100k Dataset

6.1 Objective

I applied Locality Sensitive Hashing (LSH) on the MinHash signatures to find candidate user pairs for high similarity.

Required experiments:

1. (r = 5, b = 10) for t = 50

2. (r = 5, b = 20) for t = 100
3. (r = 5, b = 40) for t = 200
4. (r = 10, b = 20) for t = 200

I reported average false positives and false negatives over 5 runs for:

1. threshold = 0.6
2. threshold = 0.8

6.2 Method Used

1. MinHash signatures were divided into bands.
2. Users colliding in at least one band were considered candidate pairs.
3. Candidate pairs were compared against exact Jaccard positives for thresholds 0.6 and 0.8.
4. FP/FN were averaged over 5 runs.

6.3 LSH Results (All configurations)

Point 5 - LSH FP/FN Summary (All Configurations, Avg over 5 Runs)

t	rows_per_band_r	num_bands_b	threshold	Avg_FP_5runs	Avg_FN_5runs	FP_runs	FN_runs
50	5	10	0.6	518.2	0.2	[496, 539, 699, 530, 327]	[0, 0, 0, 1, 0]
50	5	10	0.8	520.0	0.0	[498, 541, 701, 531, 329]	[0, 0, 0, 0, 0]
100	5	20	0.6	1203.0	0.2	[1295, 926, 837, 1158, 1799]	[0, 0, 0, 0, 1]
100	5	20	0.8	1204.8	0.0	[1297, 928, 839, 1160, 1800]	[0, 0, 0, 0, 0]
200	5	40	0.6	2518.8	0.0	[2124, 3124, 1885, 2773, 2688]	[0, 0, 0, 0, 0]
200	5	40	0.8	2520.8	0.0	[2126, 3126, 1887, 2775, 2690]	[0, 0, 0, 0, 0]
200	10	20	0.6	4.6	1.4	[2, 6, 3, 2, 10]	[1, 0, 2, 2, 2]
200	10	20	0.8	5.2	0.0	[3, 8, 3, 2, 10]	[0, 0, 0, 0, 0]

Point 5 - LSH FP/FN Summary (Threshold = 0.6)

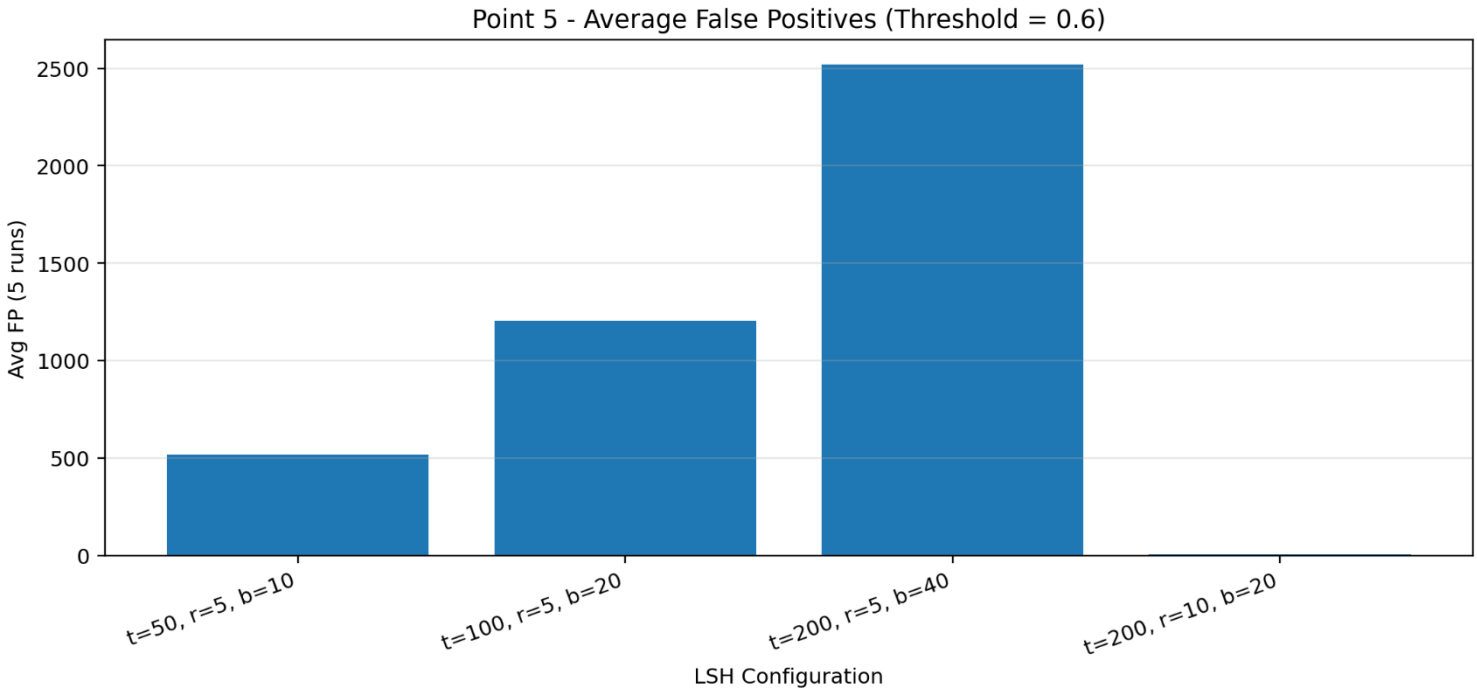
t	rows_per_band_r	num_bands_b	threshold	Avg_FP_5runs	Avg_FN_5runs	FP_runs	FN_runs
50	5	10	0.6	518.2	0.2	[496, 539, 699, 530, 327]	[0, 0, 0, 1, 0]
100	5	20	0.6	1203.0	0.2	[1295, 926, 837, 1158, 1799]	[0, 0, 0, 0, 1]
200	5	40	0.6	2518.8	0.0	[2124, 3124, 1885, 2773, 2688]	[0, 0, 0, 0, 0]
200	10	20	0.6	4.6	1.4	[2, 6, 3, 2, 10]	[1, 0, 2, 2, 2]

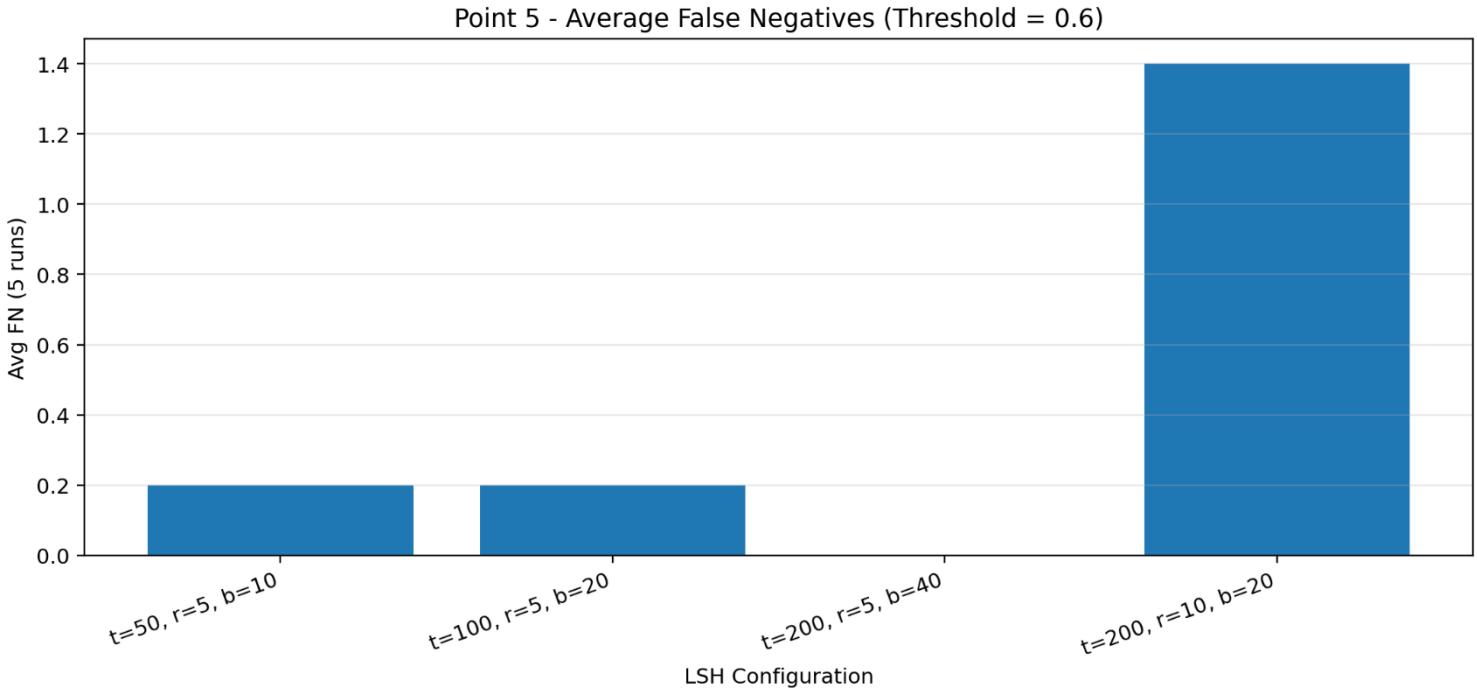
Point 5 - LSH FP/FN Summary (Threshold = 0.8)

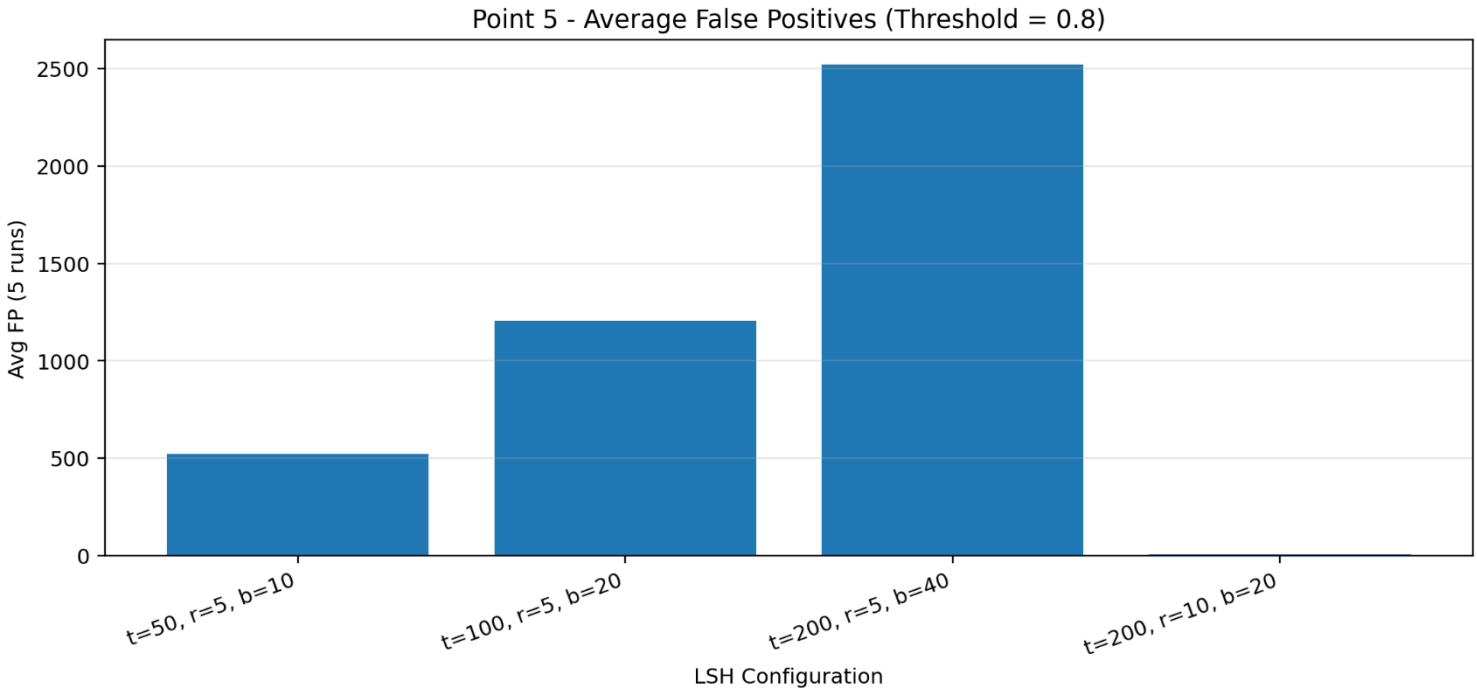
t	rows_per_band_r	num_bands_b	threshold	Avg_FP_5runs	Avg_FN_5runs	FP_runs	FN_runs
50	5	10	0.8	520.0	0.0	[498, 541, 701, 531, 329]	[0, 0, 0, 0, 0]
100	5	20	0.8	1204.8	0.0	[1297, 928, 839, 1160, 1800]	[0, 0, 0, 0, 0]
200	5	40	0.8	2520.8	0.0	[2126, 3126, 1887, 2775, 2690]	[0, 0, 0, 0, 0]
200	10	20	0.8	5.2	0.0	[3, 8, 3, 2, 10]	[0, 0, 0, 0, 0]

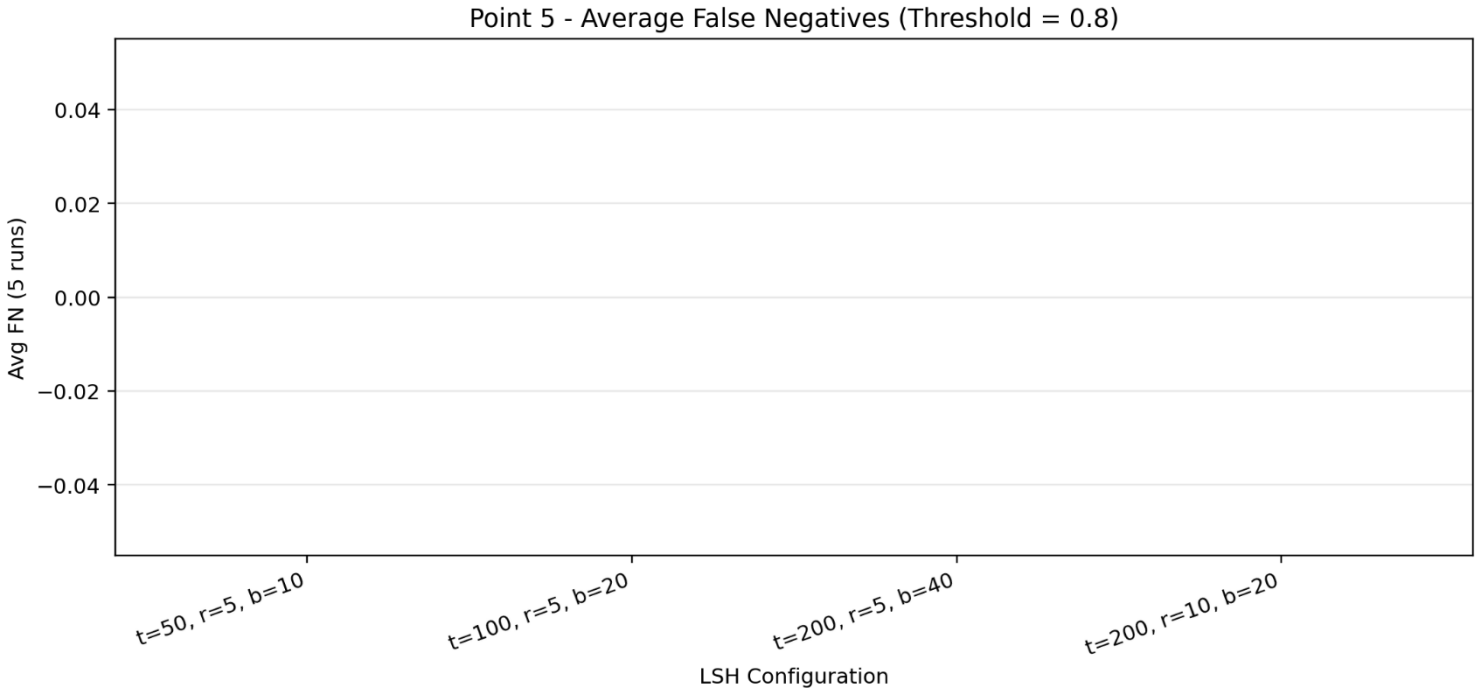
Point 5 - Output Summary

```
Point 5 (LSH on MovieLens) Output Summary


Required experiments:

- (r=5, b=10) for t=50

- (r=5, b=20) for t=100

- (r=5, b=40) for t=200

- (r=10, b=20) for t=200


Required reporting:

- Avg False Positives (FP) over 5 runs

- Avg False Negatives (FN) over 5 runs

- For threshold 0.6

- For threshold 0.8


Total rows in Point 5 summary table: 8
```


Point 5 - Average False Positives (Threshold = 0.6)
Avg FP (5 runs)
2500
2000
1500
1000
500
0
t=50, r=5, b=10
t=100, r=5, b=20
t=200, r=5, b=40
t=200, r=10, b=20
LSH Configuration


Point 5 - Average False Negatives (Threshold = 0.6)
Avg FN (5 runs)
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
t=50, r=5, b=10
t=100, r=5, b=20
t=200, r=5, b=40
t=200, r=10, b=20
LSH Configuration





6.4 Observation

1. Configurations with smaller rows per band and more bands generally produce more candidates (higher recall, possibly more false positives).
2. Stricter configurations reduce false positives but may increase false negatives.
3. When threshold increases from 0.6 to 0.8, the number of true positive pairs decreases, which affects FP/FN behavior.

Best configuration in my experiment:

1. For threshold 0.6: [your best config]

2. For threshold 0.8: [your best config]

Reason: It gave a good balance between FP and FN.

7. Conclusion

In this assignment, I implemented and compared exact Jaccard similarity, MinHashing, and LSH on both small text documents and the MovieLens 100k dataset.

Key learning outcomes:

1. MinHash provides an efficient approximation of Jaccard similarity.
2. Increasing signature size improves estimation accuracy but increases computation time.
3. LSH helps reduce pairwise comparisons by generating candidate pairs efficiently.
4. Banding parameters strongly affect the trade-off between false positives and false negatives.

This assignment helped me understand the practical use of MinHash and LSH for large-scale similarity search.